

初识 AJX Builder

1. 概述

Ajx Builder 本质上是一个 CLI，把基于 AJX 环境开发的资源构建成

`.ajx` 文件供引擎读取。通过读 Builder 的源码可以让我们对整个 AJX 构建流程有清晰的认知，横向解决业务问题时可以从编译器角度出发看是否有基于编译层面的切面解决方案，为业务提效。

2. 知识点储备

- [gulp](#)：自动化的构建工具，通过自定义的 task 把开发者从繁重的编译流程中解放出来
- [babel](#)：著名的开源 JavaScript 编译器、转译器。感兴趣的同学也可以看下我写的 [《充电系列：Babel 解析》](#)
- [terser](#)：JS 代码压缩器，webpack 中也用到了这个
- [AST](#)：抽象语法树，编译原理中非常重要的知识点。不论是 babel 还是 ts 自带的编译器都离不开对 AST 的操作
- 编译相关知识点：《编译原理》——阿尔弗雷德·艾侯
- CLI 相关：参考 Taro 团队的掘金小册-[CLI 原理及不同端的运行机制](#)
 - [tj/commander.js Node.js](#) - 命令行接口全面的解决方案，灵感来自于 Ruby's [commander](#)。可以自动的解析命令和参数，合并多选项，处理短参等等，功能强大，上手简单
 - [jprichardson/node-fs-extra](#) - 在 Node.js 的 fs 基础上增加了一些新的方法，更好用，还可以拷贝模板

3. 环境准备



复制代码

```
git clone git@gitlab.alibaba-inc.com:AMAP_AJX/AJX3_builder.git && git checkout v7.1.2
```

4. 架构及目录解析

4.1 架构图

编译器架构图



4.2 目录解析

注：尾部标【非核心代码】的是与核心流程无关的。

复制代码

```
|— README.md
|— build
|   |— api // 编译前安装 Bundle 依赖等 api 行为 非核心代码
|   |— bin
|       |— ajx.js // ajx3 环境变量的入口文件，此文件在 package.json ->
bin 进行了配置 核心代码
|       |— gotoVersionControl.js // ajx3 CLI 版本控制逻辑 非核心代码
|   |— cmd // builder 编译完后调用 cmd 下的二进制库生成 .ajx 文件 非核心代
码
|   |— common // 一些工具类 非核心代码
|   |— config.js // 记录了全局的配置。比如 process.cwd() 这种路径 非核心代
码
|   |— context.js // 编译器插件化提供给业务插件的 context 资源 非核心代码
|   |— esconfig_new.js // 编译过程中基于 eslint 的规则校验 非核心代码
|   |— esconfig_tesla.js // Tesla 平台的校验规则 非核心代码
|   |— eslint_rules_normal.json // 普通模式的 eslint 规则 非核心代码
|   |— eslint_rules_strict.json // 严格模式的 eslint 规则 非核心代码
|   |— index.js // ajx.js 的命令分发到此文件中 核心代码
|   |— lib // 一些工具类 非核心代码
|       |— pageOption.js // 核心工具类，封装了通过文件名获取文件绝对路径的方
法 核心代码
|       |— util
```

	└─ plugins // 插件组，内部集成了基于 <code>babel</code> 的 <code>jsx</code> <code>less</code> 等文件的编译规则
	└─ fast_refresh // 快速刷新相关 非核心代码
	└─ ajx_page_code.js
	└─ fast_refresh.js
代码	└─ file_compress.js // <code>js</code> 压缩，内部使用了 <code>terser</code> 进行压缩 非核心
	└─ lint_code.js
	└─ lint_fix.js
	└─ loader_axml.js // <code>axml</code> 转换器
	└─ loader_axss.js // <code>.axss</code> 转换器
	└─ loader_es.js // <code>es</code> 转换器
	└─ loader_jsx.js // <code>jsx</code> 转换器
	└─ loader_less.js // <code>less</code> 加载器
	└─ loader_less_plugins // <code>less</code> 文件加载器
	└─ colorize.js
	└─ index.js
	└─ loader_ts.js
	└─ loader_tsx.js
	└─ tasks // <code>gulp</code> 的各种 <code>Task</code> 核心代码
	└─ banner.js //根据配置的 <code>banner</code> 信息，插入字符串到对应的文件中
	└─ bundle.js // <code>Bundle</code> 相关工具类，比如版本号对比等
	└─ bundle_filter.js // 过滤 <code>framework</code> 基础库，提高编译速度
	└─ bundle_protocol.js // 把 <code>Bundle</code> 的 <code>bundle.json</code> 转化成对象供后
续流程使用	
	└─ check_preload_config.js // 预加载配置校验
	└─ code_clean.js // 删除代码中的 <code>ajx.log.print</code> 和 <code>ajx.log.toast</code>
调用	
	└─ common_lib_path_map.json
	└─ compile // 编译
	└─ compile_native.js // 基于 <code>stream</code> 的编译流程 核心代码
	└─ create_compile_stream.js // 工具类
	└─ index.js // 编译入口文件分发逻辑
	└─ config
	└─ index.js // 工具类
	└─ create_axj_file.js // 调用上述的 <code>ajxGenor</code> 把 <code>js</code> 、资源文件加密
成 .ajx 文件	
	└─ create_all_test.js // 测试用例
	└─ create_server.js // 本地预览服务
	└─ filter_pages.js // 过滤排除的 <code>page</code> 文件不进行编译
	└─ fix_filename.js // 拆 <code>bundle</code> 做的工具，与编译无关
	└─ image_3x.js // 图片逻辑处理
	└─ load_depend_tree.js // <code>gulp</code> 的 <code>task</code> 用于从 <code>page</code> 为根节点加载依
赖树	
	└─ moduleloader.js // 核心加载器，供 <code>load_depend_tree.js</code> 调用
	└─ preload_bundle.js // 加载云化的 <code>bundle</code>

```

|   |   |— project_resource.js // 搜集Bundle中的所有文件 -> Object; key
为文件名, value 为文件全路径
|   |   |— protocol // 一些协议相关的代码 非核心代码
|   |   |— tsconfig.json
|   |   |— tslint.json
|— gulpfile.js // gulp 配置文件, 供 index.js 文件调用, 配置了 gulp 的
task 列表
|— index.js
|— jest.config.js // jest 相关配置
|— jest.setup.js
|— lib // 一些 lib 库, 内部包含了 Bundle 全局的依赖比如 Promise 以及快速预
览的 html 资源
|— package.json
|— script // Builder 相关辅助脚本, 比如发布等
|— test // Builder 的测试用例

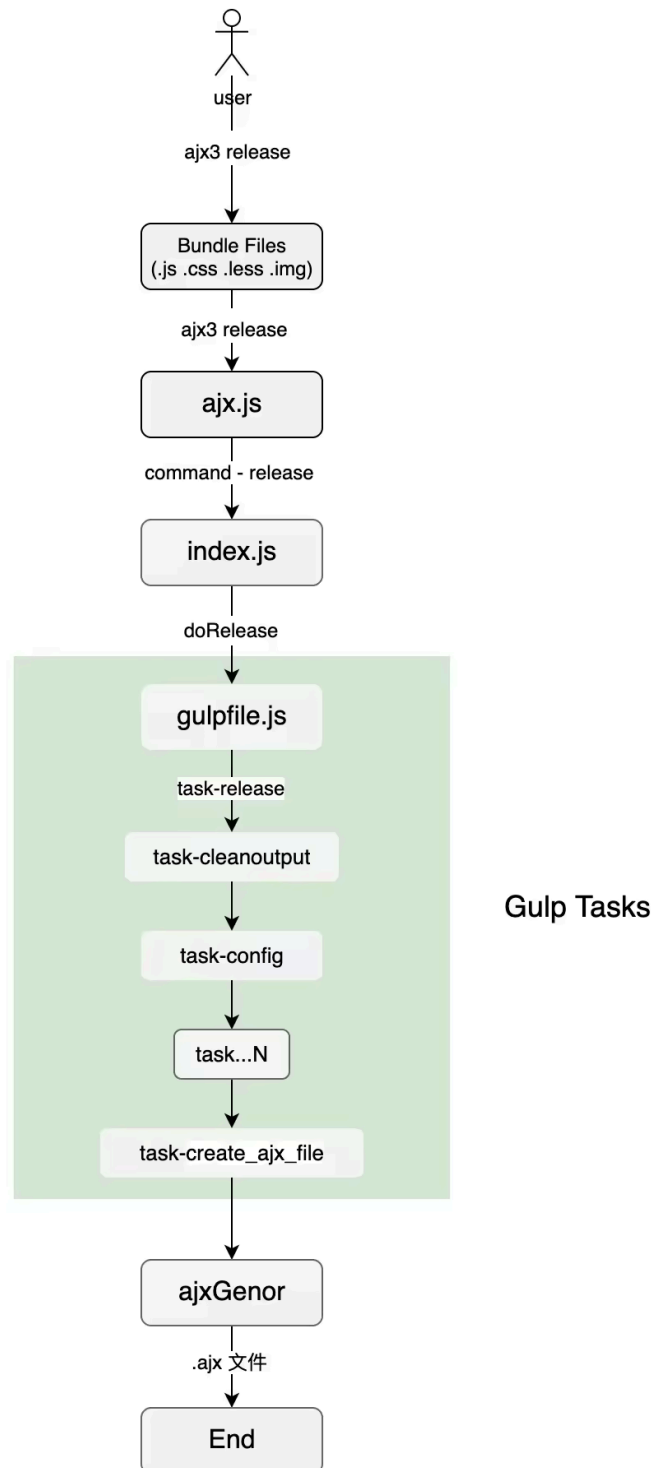
```

5. 核心流程解析

梵天打包时执行的是 Builder 的

`ajx3 release` 命令, 下面的核心流程我们以此命令为主线进行分析。

5.1 流程图



5.2 流程解析

1. 用户在 Bundle 根目录下输入 ajx3 release 命令

用户输入的 ajx3 全局命令由于已经在 package.json 中进行了配置，所以会调用到 ajx.js 中的 program.command('release') 下，由于我们没有在 release 命令后加参数，所以此 command 会被 commander 库分发到 action 下，并调用 ajxBulde 的 release 方法。

```
JS ajax.js ×
build > bin > JS ajax.js > ...
196 .option('--port <port>', '设置打包端口号')
197 .option('--tests <false>', '是否打包测试用例')
198
199 .option('--bundle-param <json str>', '注入 bundle.json 参数')
200 .option('--not-clean-log', '不删除 ajax.log.print 及 ajax.log.toast')
201 .option('--skipCodeCheck', '跳过代码检查')
202
203 // 指定文件编译
204 .option('--file <file>', '指定编译: 匹配规则 /dir/**/a/f.js')
205 .option('--fileCfg <fileCfg>', '指定编译: 匹配规则在配置文件里')
206
207 .action(function() {
208     cmd = 'release';
209     if (this.preview_qrcode == undefined) {
210         this.preview_qrcode = false;
211     }
212     if (this.disableAutoOpenBrowser == undefined) {
213         this.disableAutoOpenBrowser = false;
214     }
215
216     onlineLog('cli', 'release', process.argv.slice(2));
217     var ajxBuild = require('../index');
218     ajxBuild['release']({
219         'enable_preview_qrcode': false,
220         'enable_release_qrcode': this.preview_qrcode,
221         'enable_preview_browser': false,
```

2. gulpfile.js 执行 release 的 task

经过流程 1 的分发，代码调用到 gulpfile.js 中的 release 任务，开始交给 gulp 框架顺序执行子任务列

3. gulpTask: config

解析 Bundle 根目录下的 ajax-config.js 作为用户指定的编译条件，储存到 config.js 全局对象中

4. gulpTask: plugin-start

用户可以自定义编写编译器插件，编译器插件可以在 start 与 done 的编译周期中获得回调做自定义的事情，plugin-start 这个任务就是 start 生命周期。详细参考：

<http://30.28.11.32:8888/server2/vpage?id=886cdfd7759b6f55515714f88a6171d6>

5. gulpTask: load_all_page

收集所有待编译的 page 文件存储到 gulpfile.js 中的 wait_compile_pages 数组中供后续任务使用

```
JS ajax.js JS index.js build gulpfile.js x JS index.js .../config
gulpfile.js > gulp.task('load_all_page') callback > forEach() callback
task_load_depend_tree(wait_compile_pages, lld);
364 });
365
366 // 编译所有配置中的页面
367 gulp.task('load_all_page', function (done) {
368   timeLog.start('task:load_all_page');
369   if (config.isPreloadBundle(config.bundleOptions)) {
370     const res = preloadBundle.checkFileType(pages);
371     if (!res.result) {
372       logger.stop(new Error(res.message));
373       return;
374     }
375   }
376
377   pages.forEach(function (itemPage) {
378     wait_compile_pages.push(itemPage);
379   });
380   if (!config.isRelease) {
381     getBundleRouteFiles().forEach((item) => wait_compile_pages.push({main: item}));
382   }
383   timeLog.start('end:load_all_page');
384   done();
385 });
386
```

6. gulpTask: load_depend_tree

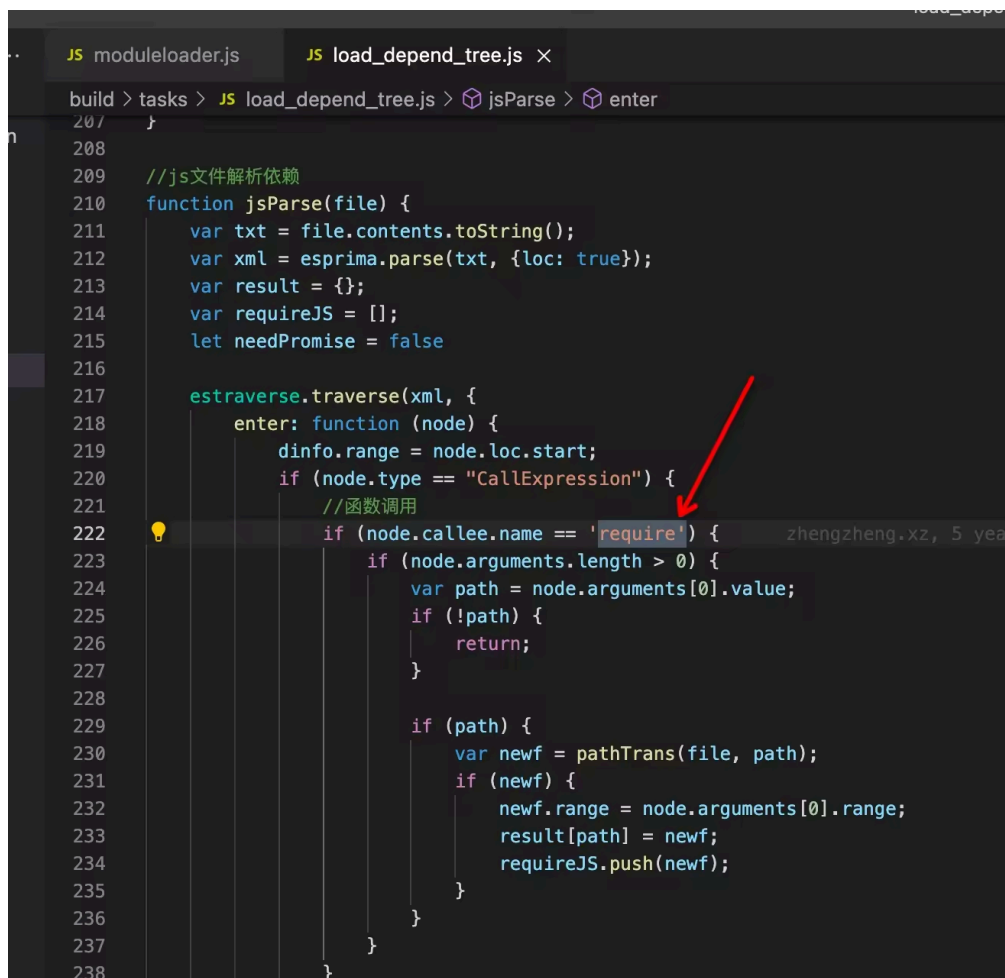
核心流程!!! 以 page 为根节点, 搜集 page 依赖的所有文件。

文件转换过程: 在加载文件之前会进行文件转换操作把 *.jsx *.page 转换成 *.js。此过程会把 es6 语法转成 es5 语法。

```
... JS moduleloader.js x
build > tasks > JS moduleloader.js > loader > _transform > lld
son
146   _transform: function (vf) {
147
148     var extname = path.extname(vf.path);
149     var lld = this.getloader(extname);
150     while (lld) {
151       vf = lld(vf);
152       var newextname = path.extname(vf.path);
153       if (newextname != extname) {
154         extname = newextname;
155         lld = this.getloader(extname);
156       }
157       else {
158         lld = null;
159       }
160     }
161
162     return vf;
163   },
164
165   load(abpath) {
166     return fileCache.get(abpath);
167   },
168
```

依赖搜集过程: 以 *.js 文件为例, load_depend_tree.js 内部调用了 jsParse 方法通过 esprima 库遍历 AST, 由于经过了文件转换过程, 所有的 import 都被转成了 require, 因此依赖搜集过程只需要

找 'require' 的语句就可以了，找到后的所有依赖会被压到 requireJS 数组中进而存储到 moduleloader.js 中的 fileCache Map 中缓存起来。



```
JS moduleloader.js JS load_depend_tree.js X
build > tasks > JS load_depend_tree.js > jsParse > enter
207 }
208
209 //js文件解析依赖
210 function jsParse(file) {
211     var txt = file.contents.toString();
212     var xml = esprima.parse(txt, {loc: true});
213     var result = {};
214     var requireJS = [];
215     let needPromise = false
216
217     estraverse.traverse(xml, {
218         enter: function (node) {
219             dinfo.range = node.loc.start;
220             if (node.type == "CallExpression") {
221                 //函数调用
222                 if (node.callee.name == 'require') {
223                     if (node.arguments.length > 0) {
224                         var path = node.arguments[0].value;
225                         if (!path) {
226                             return;
227                         }
228
229                         if (path) {
230                             var newf = pathTrans(file, path);
231                             if (newf) {
232                                 newf.range = node.arguments[0].range;
233                                 result[path] = newf;
234                                 requireJS.push(newf);
235                             }
236                         }
237                     }
238                 }
239             }
240         }
241     });
242 }
```

7. gulpTask: compile

核心流程!!! 内部调用了 compile_pages task 执行编译。

compile_pages 内部通过 stream 先后对图片、资源、测试代码注入、性能插桩、代码压缩、sourMap等功能进行了能力实现

```
compile_native.js -
JS compile_native.js x
build > tasks > compile > JS compile_native.js > <unknown> > exports > <function>
33
34 module.exports = function (outputRelativeDir, wait_compile_pages) {
35   var transform_font_clor = require('.../plugins/transform_font_color.js');
36   return function (stream) {
37     const isPreload = config.isPreloadBundle(config.bundleOptions);
38
39     return stream
40
41       .pipe(add3xImg())
42
43       // 必须在 add3xImg 后执行
44       .pipe(web3.parse3xOutPath())
45
46       // transform_font_clor
47       .pipe(transform_font_clor())
48
49       // handle process.env.NODE_ENV
50       .pipe(transform_node_env())
51
52       //资源定位
53       .pipe(reloc())
54
55       //代码注入
56       .pipe(config.clean_log? codeclean():gutil.noop())
57
58       //函数性能插桩
59       .pipe(codeInstrument())
60
61       //performance代码注入
62       .pipe(performanceInject())
63
64       //检查代码和 bundle.json 中的 URI 是否都合法 叶俊星(达诺), 3 years ago * 1
65       .pipe(protocolCheck())
66
67       // 执行插件: steps.compress_before
68       .pipe(custom_steps_plugin('compress_before'))
69
70       //压缩(xml、css、js)
71       .pipe(config.enable_compress ? compress() : gutil.noop())
72
73       // 执行插件: steps.compress_after
74       .pipe(custom_steps_plugin('compress_after'))
75
76       .pipe(config.enable_compress ? checkForbiddenKeywords(): gutil.noop())
77
78       // .pipe(fastRefresh.addHotCode()) // 功能未实现, 后面再支持
79
80       .pipe(addBanner())
```

8. gulpTask: create_ajax_file

调用 build/cmd 下的二进制文件对 Bundle 资源进行加密、整合。形成最终的 .ajax 文件

9. gulpTask: plugin-done

同 4，都是调用业务插件的生命周期

6. 思考

1. 如果有横向的切面诉求，可以考虑自定义编译器插件

- 关于如果开发编译器插件参考: <http://30.28.11.32:8888/server2/vpage?id=886cdfd7759b6f55515714f88a6171d6>
- 关于插件的调用流程参考 5.2.4 与 5.2.9